

The OSI model — how all networked devices communicate

What is the OSI model?

- OSI = **Open Systems Interconnection** model. A framework of rules for how devices send, receive, and interpret data
- Every device follows it — so a Mac can talk to a Windows PC, a phone can talk to a server
- Made of **7 layers**. Data travels down the layers when sending, and up the layers when receiving
- As data travels through each layer, extra information is added — this is called **encapsulation**. Removing it on the other end = **decapsulation**

Easy way to remember it

Think of sending a letter in an envelope

You write the letter (your data at Layer 7). Each layer wraps it in something new — an envelope, a shipping label, a tracking number. By the time it leaves, the letter is buried inside layers of packaging. When it arrives, each layer is unwrapped in reverse until the original letter is revealed.

Memory trick for all 7 layers (top to bottom)

"All People Seem To Need Data Processing"

Application → Presentation → Session → Transport → Network → Data Link → Physical

The 7 OSI layers — what each one does

Layer 7 is closest to the user. Layer 1 is the physical cable. Data travels 7 → 1 when sending, 1 → 7 when receiving.

7 Application

The layer YOU see. Email clients, browsers, file browsers. Protocols like DNS (translates website names into IP addresses). This is where you interact with data.

6 Presentation

Translates data into a standard format so different software can understand it. Handles encryption — HTTPS happens here. So two different email apps can still read the same email.

5 Session

Opens and manages the connection (called a session) between two devices. Adds checkpoints — if data is lost, only the most recent piece needs to be re-sent, not everything from the start.

4 Transport

Decides HOW data travels — reliably (TCP) or fast (UDP). Breaks data into chunks, adds error checking. This is where the big decision between TCP and UDP happens.

3 Network

Routing — finds the best path to send data across networks using IP addresses. Routers operate here. Protocols: OSPF and RIP. This layer deals with IP addresses.

2 Data Link

Physical addressing. Takes the IP address from Layer 3 and adds the MAC address of the receiving device. Formats the data for transmission. Frames live here.

1 Physical

The actual hardware — ethernet cables, network cards, signals. Data is converted into electrical signals (binary: 1s and 0s) and physically transmitted.

Security note: Attacks can happen at any layer. Layer 1 = cutting a cable. Layer 2 = MAC spoofing. Layer 3 = IP spoofing. Layer 7 = phishing. Knowing the layer helps you know what kind of attack you're dealing with.

Packets and frames — the difference

What is a packet?

→ A **packet** = a small chunk of data at **Layer 3** (Network). Contains an IP header and the actual data (payload)

→ Data is broken into packets so it can travel across networks efficiently — less chance of bottlenecking than sending everything at once

→ When you load an image, it arrives as many small packets, then your device reconstructs the full image

What is a frame?

→ A **frame** = the wrapper around a packet at **Layer 2** (Data Link). Adds MAC addresses around the outside

→ The frame is the envelope. The packet is the letter inside. When the envelope is opened (frame stripped away), what's left is the packet

Key headers inside a packet

TTL

Time to Live — an expiry timer. If a packet gets lost and keeps bouncing around a network, TTL counts down and kills it so it doesn't clog things forever

CHK

Checksum — a calculated number. If the data changes while travelling (gets corrupted), the checksum won't match and the device knows it's corrupt

SRC

Source address — where the packet came from. Where to send the reply back

DST

Destination address — where the packet is going next

TCP vs UDP — the two ways data travels

Both are Layer 4 protocols. They decide whether data is sent reliably or fast. You pick one based on what matters more for that situation.

TCP — Transmission Control Protocol

Reliable. Checks everything arrived. Slower.

Advantages

Guarantees data arrives correctly and in order. Both devices sync so neither gets flooded. Has error checking.

Disadvantages

Slower. Needs a stable connection. If one tiny chunk is missing, nothing can be used until it's re-sent.

Used for: file downloads, browsing, email — when accuracy matters

UDP — User Datagram Protocol

Fast. Sends and hopes for the best. No checking.

Advantages

Much faster than TCP. Flexible for developers. No connection setup needed.

Disadvantages

Doesn't care if data arrives. No guarantee. Bad connection = terrible experience for the user.

Used for: video streaming, voice calls, ARP, DHCP — when speed matters more than perfection

Memory trick: TCP = certified mail (you get confirmation it arrived). UDP = dropping a flyer in someone's letterbox (you hope they get it, but you're not checking).

TCP three-way handshake — how a connection is established

Why does TCP need a handshake?

→ TCP is **connection-based** — before any data is sent, the two devices must agree they are both ready

→ This agreement is called the **three-way handshake** — three messages to establish the connection

The steps

SYN

Step 1 — Client sends SYN

"Hey, I want to connect. Here's my sequence number so we can keep track of data order." Synchronise = SYN.

SYN/ACK

Step 2 — Server replies SYN/ACK

"Got your request. Here's MY sequence number. I acknowledge (ACK) yours too." Two things in one message.

ACK

Step 3 — Client replies ACK

"I acknowledge your sequence number. Connection established. Send me data." Now both sides are synced and ready.

DATA

Actual data starts flowing

After the handshake, the real data (files, web pages, etc.) starts being sent in numbered packets.

FIN

Closing the connection cleanly

When done, one device sends FIN. The other acknowledges and sends FIN back. Both confirm. Connection closed properly.

RST

Emergency close

If something goes wrong (crash, error, low resources), RST abruptly ends everything — no clean close. Last resort.

Security note — SYN flood attack: An attacker sends thousands of SYN packets but never completes step 3. The server reserves resources for each half-open connection and gets overwhelmed. This is a type of DoS (Denial of Service) attack.

TCP/IP model — the simplified version of OSI

What is TCP/IP?

→ The TCP/IP model is a **4-layer** version of the 7-layer OSI model — a summarised, practical version of the same idea

→ Data still gets encapsulated as it moves through layers. Still decapsulated when received on the other end

The 4 layers compared to OSI

A Application

Combines OSI layers 5, 6, and 7. This is everything the user sees — browsers, email, DNS, etc.

T Transport

Same as OSI Layer 4. TCP and UDP both live here.

I Internet

Same as OSI Layer 3. IP addresses and routing.

N Network Interface

Combines OSI layers 1 and 2. Physical hardware and MAC addresses.

Memory trick: TCP/IP = A TIN. Application, Transport, Internet, Network Interface. Four layers vs OSI's seven — same concept, just grouped differently.

Key TCP packet headers to know

- **Source port** — randomly chosen port the sender opens (0–65535)
- **Destination port** — the specific port the service is running on (e.g. port 80 for web)
- **Sequence number** — keeps track of packet order so data is reassembled correctly
- **Acknowledgement number** — confirms which sequence number was received (sequence + 1)
- **Checksum** — verifies the data wasn't corrupted during travel
- **Flag** — tells the device how to handle the packet (SYN, ACK, FIN, RST, etc.)

Ports — where data enters and exits a device

What is a port?

- Ports are numbered doors on a device. Data coming in or going out must go through a specific port
- Port numbers range from **0 to 65,535**
- Ports 0–1024 are called **common ports** — reserved for well-known services
- Every service has a standard port number — so any browser or client knows exactly where to knock

Easy way to remember it

Think of a harbour with numbered docks

Ships (data) can only dock at the right port. A cruise ship can't dock at a fishing port — wrong size, wrong facilities. In networking, a web browser uses port 80, not port 22 (that's for SSH). If you show up at the wrong port, nothing happens.

Common ports to memorize

21 FTP File Transfer Protocol — downloading files from a server

22 SSHSecure Shell — logging into systems remotely via text/command line (encrypted)

80 HTTPHyperText Transfer Protocol — regular web browsing (not encrypted)

443 HTTPSHTTP Secure — same as HTTP but encrypted. The padlock in your browser.

445 SMBServer Message Block — file and printer sharing across a network

3389 RDPRemote Desktop Protocol — visually logging into another computer remotely (GUI)

Security note: Ports are one of the first things a hacker scans when attacking a network. Finding port 22 open = there's an SSH service. Finding port 3389 open = remote desktop is running. Knowing which ports are open tells an attacker what's possible to attack. Port scanning is a real reconnaissance technique.

Remember: these port numbers are just the standard defaults. You can run a web server on port 8080 instead of 80 — but then you have to specify it in the URL: <http://example.com:8080>. Applications assume the standard unless told otherwise.

Quick-reference summary

- ★ OSI model = 7-layer framework for how all networked devices communicate
- ★ Memory trick for layers: "All People Seem To Need Data Processing" (7 → 1)
- ★ Encapsulation = wrapping data with extra info at each layer. Decapsulation = unwrapping on arrival
- ★ Packet = data chunk at Layer 3 (has IP address). Frame = wrapper at Layer 2 (has MAC address)
- ★ TCP = reliable, slow, checks everything. Used for files, email, browsing
- ★ UDP = fast, no checking, doesn't care if data arrives. Used for video, voice, ARP, DHCP
- ★ TCP handshake: SYN → SYN/ACK → ACK (then data flows, ends with FIN)
- ★ RST = emergency close. SYN flood = attacker sends SYN without completing the handshake
- ★ TCP/IP model = 4 layers: Application, Transport, Internet, Network Interface (A TIN)
- ★ Ports 0–1024 = common ports. Key ones: 21 FTP, 22 SSH, 80 HTTP, 443 HTTPS, 445 SMB, 3389 RDP
- ★ Open ports = attack surface. Attackers scan ports to find what services are running